PRODUCT DOCUMENTATION

AI-Powered Autonomous Database Administrator

End-to-end product reference: architecture, components, API, data model, deployment, security & operations

Swapneel Kulkarni
Version 1.0 swapneel2121@gmail.com

Contents

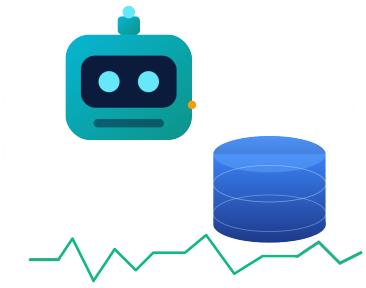
1. Overview

The **AI-Powered Autonomous Database Administrator** is an agent that continuously monitors PostgreSQL (and MySQL) databases, detects performance problems in real time, and proposes — and, with human approval, safely applies — optimizations. It combines asynchronous metric collection, statistical anomaly detection, query fingerprinting, both rule-based and LLM-powered SQL optimization, workload replay against a shadow database, predictive capacity planning, security-anomaly detection, and a natural-language interface — all surfaced through a real-time React dashboard.

This document is the end-to-end product reference: architecture, every component and capability, the REST API, the data model, deployment and operations, configuration, security, observability, a requirements traceability matrix against the project brief, and the known limitations / roadmap.

Design principles

- **Safety first.** Every destructive change is gated by human-in-the-loop approval, validated by workload replay, recorded in an immutable audit log, and reversible via rollback.
- **Low overhead.** Async polling with a small connection ceiling and tunable intervals targets the brief's <2% overhead budget on the monitored database.
- **Graceful degradation.** The system boots and serves even when TimescaleDB, Redis, Ollama, or Docker are unavailable, retrying dependencies and falling back to rule-based logic.
- **Containerized and reproducible.** The entire stack — nine services — comes up with a single `docker compose up`.



2. System Architecture

The agent runs a closed-loop pipeline. Monitored databases are polled by an asynchronous monitoring agent; metrics and normalized query statistics are persisted to TimescaleDB and served by a FastAPI application (REST + Server-Sent Events). Analysis components (fingerprinting, anomaly detection, EXPLAIN parsing, security scanning) feed action components (LLM/rule-based optimizer, workload replay, capacity forecaster, approval state machine, backup verification, notifications), which the React dashboard renders live.

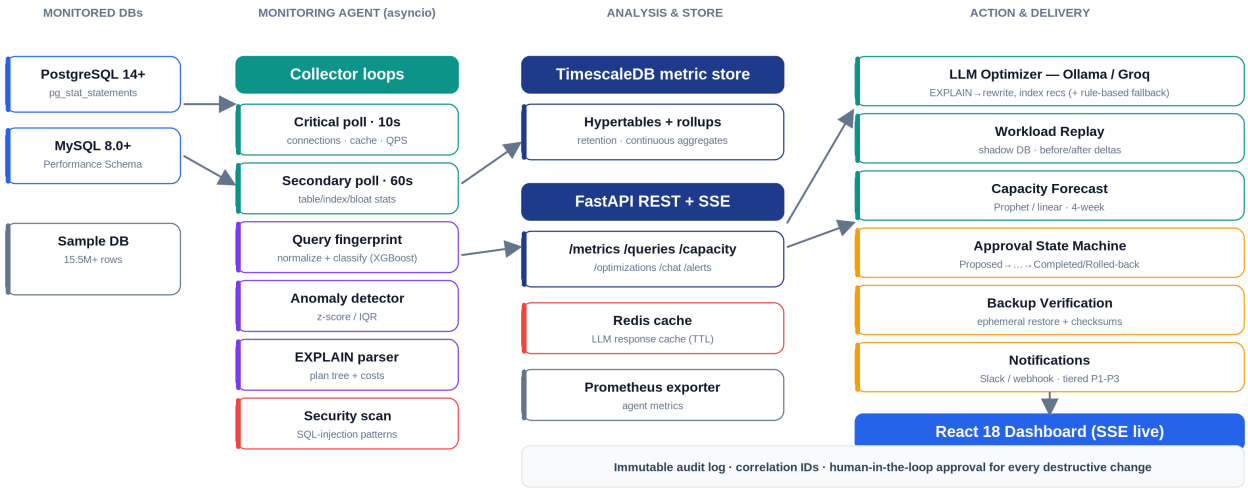


Figure 1 — End-to-end optimization pipeline and component map.

Layers

Layer	Responsibility	Key components
Monitored DBs	Targets being observed	PostgreSQL 14+, MySQL 8.0+, bundled sample DB
Monitoring agent	Async polling & analysis	Collector loops, fingerprint, anomaly (z-score/IQR), EXPLAIN parser, security scan
Store & API	Persistence & serving	TimescaleDB, FastAPI REST + SSE, Redis cache, Prometheus exporter
Action & delivery	Optimize, validate, deliver	LLM/rule optimizer, workload replay, capacity forecast, approval FSM, backup verify, notifications, React dashboard

3. Core Capabilities

3.1 Real-time monitoring

An asyncio agent polls each database on two cadences — critical metrics every 10s (active connections, max connections, cache-hit ratio, QPS computed from the transaction-counter delta, lock waits, deadlocks, replication lag) and secondary metrics every 60s (table/index/bloat statistics). Snapshots are written to TimescaleDB and fanned out to the dashboard over SSE. The agent retries connecting until a database is reachable, so it self-heals if a target starts late.

3.2 Slow-query detection

Queries above a configurable threshold are pulled from pg_stat_statements (PostgreSQL) / Performance Schema (MySQL), normalized and fingerprinted for deduplication, classified by access pattern, and stored. Statistical anomaly detection uses z-score and IQR over rolling per-metric windows.

3.3 SQL optimization (LLM + rule-based)

The optimizer analyzes a query (and optional EXPLAIN ANALYZE output) and produces issues, index recommendations with estimated impact, and optional rewrites. It uses an LLM (Ollama locally, Groq fallback) and — critically — falls back to a deterministic rule-based heuristic (SELECT *, missing index on filtered columns, leading-wildcard LIKE, Seq Scan, ORDER BY without LIMIT) so the pipeline works with no model.

3.4 EXPLAIN ANALYZE parser

Plans are parsed into a structured node tree (node type, costs, estimated vs actual rows, buffers, timing) for both rule-based heuristics and the LLM. EXPLAIN runs inside a transaction that is always rolled back.

3.5 Workload replay

Captured workload is replayed against an ephemeral shadow database with proposed changes applied; before/after performance deltas gate whether a change proceeds. (Implemented; requires Docker access to exercise.)

3.6 Predictive capacity planning

Time-series forecasts (Prophet when available, linear regression fallback) project metrics 4 weeks ahead with confidence bands and breach detection. Forecasts read the raw per-minute metrics so a chart appears within minutes of startup rather than waiting for hourly aggregates.

3.7 Human-in-the-loop approval

Optimization proposals move through a state machine: Proposed → Reviewed → Approved → Testing → Deploying → Monitoring → Completed / Rolled-Back / Rejected. Reviews require a mandatory comment; transitions emit Slack/webhook notifications and audit entries.

3.8 Backup verification

A verification cycle restores a backup into an ephemeral container and compares row counts / checksums against the live database, reporting integrity status. (Implemented; requires Docker.)

3.9 Security anomaly detection

Observed queries are scanned for SQL-injection patterns (e.g. UNION-based, comment-based, 1=1), privilege-escalation DDL, and SELECT * on sensitive tables. Matches raise tiered alerts and are exposed at /api/v1/alerts/security.

3.10 Natural-language interface

The AI Chat answers common health questions deterministically from the monitoring data (no model needed) and, with an LLM, translates free-form English into a safe read-only SQL query against the monitored database's real schema, executing it in a READ ONLY transaction and rendering the rows as a table.

4. Technology Stack

Area	Technologies
Backend	Python 3.11 (asyncio), FastAPI, asyncpg, aiomysql, SQLAlchemy, XGBoost, Prophet, transitions, structlog, Prometheus client, httpx, tenacity
Frontend	React 18, TypeScript, Vite, Recharts, @tanstack/react-query, Tailwind CSS, Server-Sent Events

Area	Technologies
Data & infra	PostgreSQL / MySQL, TimescaleDB, Redis, Docker Compose, Ollama / Groq, MinIO
Observability	structlog (JSON), Prometheus, Grafana, OpenTelemetry, immutable audit log

5. Backend Components

Module	Purpose
agent/monitor.py	Async monitoring agent, Postgres/MySQL collectors, snapshot publishing, read-only data queries, schema introspection
agent/fingerprint.py	SQL normalization, fingerprinting, access-pattern classification (XGBoost-ready)
agent/anomaly.py	z-score / IQR metric anomalies + security pattern detection
agent/explain_parser.py	EXPLAIN ANALYZE → structured plan tree
agent/optimizer.py	LLM client (Ollama/Groq), rule-based fallback, query analysis, NL answers, English→SQL
agent/replay.py	Workload replay against shadow databases
services/timescale.py	TimescaleDB metric store: schema, writes, reads, forecasting source
services/approval.py	Proposal state machine + in-memory proposal store
services/forecasting.py	Prophet / linear forecasting with breach detection
services/backup.py	Backup restore + checksum verification
services/notifications.py	Tiered Slack/webhook/PagerDuty alerts + recent-alert buffer
api/main.py	FastAPI app, lifespan, SSE generator, router mounting, Prometheus mount
models/database.py	SQLAlchemy ORM models + audit checksum trigger
utils/config.py	Pydantic settings configuration (env-driven)

6. REST API Reference

The FastAPI application serves under the `/api/v1` prefix. Interactive OpenAPI 3.0 documentation is available at `/docs` (Swagger UI) and `/redoc`. Prometheus metrics are exposed at `/metrics` and a health probe at `/health`.

Method & Path	Purpose
GET /api/v1/metrics/databases	List monitored databases with their stable hashed ids
GET /api/v1/metrics/health/{id}	Latest health snapshot for a database
GET /api/v1/metrics/timeseries/{id}	Bucketed time series for a metric (query: metric, hours, bucket)
GET /api/v1/metrics/overview	Summary health across all monitored databases
GET /api/v1/queries/slow/{id}	Top-N slowest queries
POST /api/v1/queries/analyze	Analyze a SQL query (LLM + rule-based fallback)
GET /api/v1/queries/fingerprint	Normalize and fingerprint a SQL string
GET /api/v1/optimizations/	List optimization proposals (filter: database_id, state)

Method & Path	Purpose
POST /api/v1/optimizations/	Create a proposal from a SQL query; optional workload replay
GET /api/v1/optimizations/{pid}	Fetch a single proposal
POST /api/v1/optimizations/{pid}/review	Approve or reject with a mandatory comment
POST /api/v1/optimizations/{pid}/rollback	Roll back a deployed proposal
GET /api/v1/capacity/forecast/{id}	Forecast a metric (query: metric, lookahead_days)
GET /api/v1/capacity/forecast/{id}/all	Forecast all key metrics
POST /api/v1/chat/	Natural-language interface (health answers + English→SQL data queries)
POST /api/v1/backups/verify	Trigger an async backup verification cycle
GET /api/v1/backups/history/{id}	Recent backup verification results
GET /api/v1/alerts/	Recent alerts (filter: severity)
GET /api/v1/alerts/security	Security-specific alerts (injection, privilege, exfiltration)
GET /api/v1/stream/{id}	Server-Sent Events stream of live health snapshots

7. Data Model

TimescaleDB metric store

Table / View	Type	Contents
db_health_metrics	Hypertable	Per-poll health snapshots (connections, cache, QPS, latency, locks)
db_health_hourly	Continuous agg	Hourly rollups
db_health_daily	Continuous agg	Daily rollups
query_snapshots_store	Table	Normalized query fingerprints + aggregated stats
table_stats	Hypertable	Table size, dead/live tuples, bloat ratio
index_stats	Hypertable	Index scans and sizes

Retention and compression policies are defined for the hypertables (raw 7 days, with compression), matching the brief's hot/warm/cold tiering intent.

Application ORM models

SQLAlchemy models (PostgreSQL) capture the agent's own state: **MonitoredDatabase**, **QuerySnapshot**, **OptimizationProposal**, **AuditLog** (append-only with per-row SHA-256 checksums), **AlertRule**, **Alert**, and **BackupVerification**.

Proposal state machine

Proposed → Reviewed → Approved → Testing → Deploying → Monitoring → Completed, with branches to Rejected (from Proposed/Reviewed) and Rolled-Back (from Deploying/Monitoring). Implemented as a dependency-free, fully type-checkable state machine.

8. Frontend & Dashboard

A React 18 + TypeScript single-page app (Vite build, served by Nginx with gzip and immutable asset caching). The dashboard streams live metrics over SSE and updates every ~10 seconds.

Figure 2 — Real-time health dashboard (dark theme).

Tab	Function
Dashboard	Live KPI cards and charts (connections, cache hit, QPS, latency) via SSE
Slow Queries	Top-N slow queries with one-click Analyze / Optimize
Optimizations	Proposal review with approve / reject and rollback
Capacity	Forecast charts with confidence bands and breach alerts
AI Chat	Health Q&A; + English-to-SQL data queries rendered as tables
Settings / Alerts	Thresholds, security events, recent alerts

9. Infrastructure & Deployment

The full stack is orchestrated with Docker Compose: nine services on one network, each independently configurable. The backend connects to monitored databases and its own TimescaleDB store; the frontend proxies the API and SSE through Nginx.

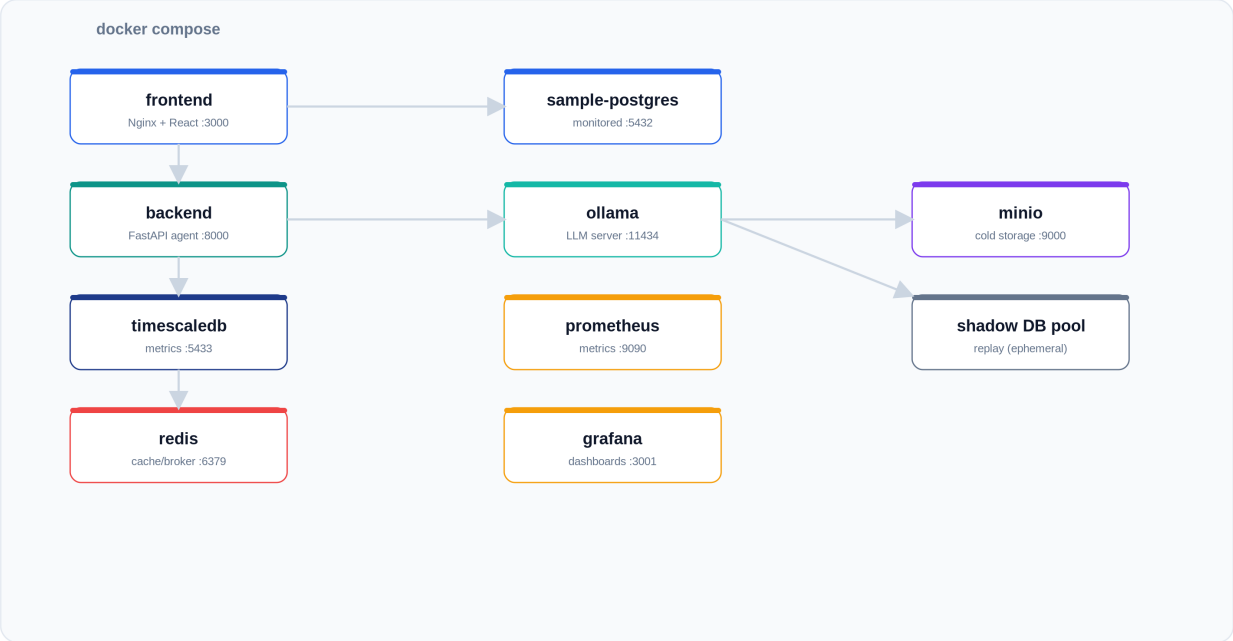


Figure 3 — Docker Compose service topology.

Service	Image	Port	Role
frontend	Nginx + React	3000	Dashboard UI
backend	FastAPI agent	8000	API, SSE, monitoring agent
timescaledb	timescale/pg16	5433	Metric store
redis	redis:7	6379	Cache / broker
sample-postgres	postgres:16	5432	Bundled monitored DB
ollama	ollama/ollama	11434	Local LLM server
prometheus	prom/prometheus	9090	Metrics scraping
grafana	grafana	3001	Agent dashboards
minio	minio	9000/9001	Cold storage

Run it locally

```
# 1) configure
cp .env.example .env

# 2) launch the full stack
cd infra/docker
docker compose up --build

# 3) (optional) enable the LLM for free-form chat
docker exec -it dba-ollama ollama pull llama3.2:1b
#    then set OLLAMA_MODEL=llama3.2:1b on the backend service and restart it
```

Endpoints once running: dashboard `localhost:3000`, API docs `localhost:8000/docs`, Grafana `localhost:3001`, Prometheus `localhost:9090`, MinIO console `localhost:9001`.

10. Configuration Reference

Configuration is environment-driven (Pydantic settings). Values resolve from the container environment (compose `environment` / `env_file`) over the baked .env. The bundled sample DB target is hard-wired in compose so monitoring works regardless of .env drift.

Variable	Default / example	Meaning
MONITORED_DBS	postgresql://...@sample-postgres:5432/sampledb	Comma-separated monitored DB URLs
TIMESCALE_URL	postgresql://...@timescaledb:5432/dba_metrics	Agent metric store
REDIS_URL	redis://redis:6379/0	Cache / broker
LLM_PROVIDER / OLLAMA_MODEL	ollama / llama3.2:1b	LLM provider and model
GROQ_API_KEY / GROQ_MODEL	(unset) / llama3-70b	Cloud LLM fallback
CRITICAL_POLL_INTERVAL	10	Critical metric poll seconds
SECONDARY_POLL_INTERVAL	60	Secondary metric poll seconds
SLOW_QUERY_THRESHOLD_MS	1 (demo) / 1000	Slow-query capture threshold
P99_REGRESSION_THRESHOLD	0.10	Auto-rollback p99 increase trigger
CAPACITY_WARNING_DAYS	28	Capacity breach warning window
SEED_USERS / SEED_ORDERS / SEED_ORDER_ITEMS	100k / 1M / 1.5M	Sample dataset size (configurable to 100M+)

11. Security

- **Read-only data queries.** English→SQL queries run inside a Postgres READ ONLY transaction with an 8s statement timeout, and are validated to be a single SELECT before execution (defense in depth).
- **Injection / anomaly detection.** Observed queries are scanned for UNION-based, comment-based and 1=1 injection patterns, privilege-escalation DDL, and SELECT * on sensitive tables; matches raise tiered alerts.
- **Human-in-the-loop gating.** Destructive changes (DROP INDEX, ALTER, migrations) require explicit approval with a mandatory comment.
- **Immutable audit trail.** Every proposed / approved / executed / rolled-back action is recorded in an append-only table with per-row SHA-256 checksums.
- **EXPLAIN safety.** EXPLAIN ANALYZE runs in an always-rolled-back transaction so analysis never mutates data.

12. Observability

- **Structured logging.** JSON logs (structlog) with correlation IDs linking monitoring → analysis → recommendation → execution.

- **Prometheus metrics.** Agent metrics exposed at /metrics for scraping; Grafana ships in the stack for dashboards.
- **Tiered alerting.** P1 → PagerDuty, P2 → Slack, P3 → email digest, routed by severity.
- **Degraded-mode visibility.** Missing dependencies are logged as warnings (TimescaleDB unavailable, prophet not installed, docker unavailable) rather than crashing startup.

13. Operations Runbook

Scenario	Resolution
Dashboard shows no metrics	Check 'collector_started' in backend logs. If 'collector_connect_failed_will_retry', the monitored DB is still seeding/unreachable — it connects automatically once ready.
Slow Queries tab empty	Requires pg_stat_statements preloaded (set in compose) and workload; recreate the sample DB volume to re-seed.
AI Chat says model unavailable	Pull a model: docker exec -it dba-ollama ollama pull llama3.2:1b; set OLLAMA_MODEL and restart backend.
Re-seed sample DB	docker compose down; docker volume rm docker_sample_pg_data; docker compose up -d --build.
Scale dataset	Raise SEED_ORDERS / SEED_ORDER_ITEMS in compose (toward 100M); larger = longer first init.

14. Requirements Traceability

Status of the project brief's EXPECTED OUTPUT items against the delivered implementation.

Expected output	Status	Notes
Web dashboard (health, alerts, recommendations, trends)	**Done**	Live SSE dashboard; alerts via API; trends in live history
Slow-query detection (thresholds, z-score/IQR, fingerprinting)	**Done**	pg_stat_statements + Performance Schema; anomaly module
LLM SQL optimization (EXPLAIN, rewrites, index recs)	**Done**	LLM + rule-based fallback
Workload replay subsystem	**Partial**	Implemented; needs Docker to exercise
Predictive capacity planning (4-week)	**Done**	Prophet + linear fallback; renders from raw metrics
HITL approval (Slack/webhook, approve/reject, rollback)	**Done**	State machine + manual rollback; auto p99 trigger partial
NL interface (Ollama/Groq, English, charts)	**Done**	Health answers + English→SQL data queries with tables
Backup verification (ephemeral restore, checksums)	**Partial**	Implemented; needs Docker; daily schedule not wired
Security anomaly detection	**Done**	Injection/privilege/exfiltration scan + /alerts/security
Documentation (diagrams, OpenAPI, runbook)	**Done**	This document + live OpenAPI at /docs

15. Known Limitations & Roadmap

The following from the brief's larger Observability / Scaling / Deployment sections are scaffolded or intentionally out of scope for a single-host development stack, and are the natural next steps:

Area	Status	Next step
Celery multi-worker coordinator	Scaffolded	Distribute monitoring across workers via Redis
PgBouncer / ProxySQL pooling	Not built	Add pooling sidecars to bound overhead
Loki / Promtail, Jaeger tracing	Scaffolded	Ship logs to Loki; wire OpenTelemetry spans
Kubernetes / Helm, Terraform, CI/CD	Not built	Production deploy manifests + pipeline
OAuth2 / RBAC, public URL	Not built	Auth0/Keycloak + roles (Admin/DBA/Viewer)
Automatic p99 rollback trigger	Partial	Wire post-deploy monitor to auto-rollback
Dedicated Alerts UI tab	Partial	Surface /api/v1/alerts in the dashboard

Appendix A. Repository Layout

```
autonomous-dba/
  backend/
    agent/      monitor, fingerprint, anomaly, explain_parser, optimizer, replay
    api/        main.py (app, SSE), routes/ (metrics, queries, optimizations,
                capacity, chat, backups, alerts)
    services/   timescale, approval, forecasting, backup, notifications
    models/     database.py (ORM), schemas.py (Pydantic)
    utils/      config.py, logging.py
  frontend/     React 18 + TS (App.tsx, hooks/useSSE, utils/api.ts)
  infra/docker/ docker-compose.yml, Dockerfile.backend, Dockerfile.frontend,
                nginx.conf, prometheus.yml, sample_pg_seed.sh
  requirements.txt  .env(.example)
```